

Data Science and Machine Learning

Fall 2022 Semester

Exam with Solutions

Instructors: V. Kantere, D. Tsoumakos

Read each question carefully, then print the letter of the correct answer on the line next to the question.

Question Set 1 (70%)

Question 1 (4%)

Let a data warehouse cube consist of 4 dimensions, each of which has 5 hierarchy levels (including all). How many cuboids exist, including the base and apex cuboids?

- a. 4^5
- b. 5^4
- c. 2^4
- d. 20

Solution

Answer: b. 5^4

For a data warehouse cube with n dimensions, where each dimension has L_i hierarchy levels (including “all”), the total number of cuboids is:

$$\prod_{i=1}^n L_i$$

Here we have 4 dimensions, each with 5 hierarchy levels:

$$5 \times 5 \times 5 \times 5 = 5^4 = 625$$

Question 2 (4%)

How is the snowflake schema different from the star schema?

- a. Snowflake does not contain a fact table
- b. Dimension tables are split into further tables in the snowflake schema
- c. Star schema has dimension tables, snowflake does not
- d. The fact table in snowflake is smaller than in star

Solution

Answer: b. Dimension tables are split into further tables in the snowflake schema

In a star schema, dimension tables are denormalized. In a snowflake schema, dimension tables are normalized, meaning they are split into multiple related tables. Both schemas have fact tables and dimension tables.

Question 3 (4%)

A data cube has n dimensions, and each dimension has exactly p distinct values in the base cuboid. Assume that there are no concept hierarchies. What is the maximum number of cells possible in the base cuboid?

- a. p^n
- b. p
- c. pn
- d. 2^n

Solution

Answer: a. p^n

The base cuboid contains the most detailed data. With n dimensions and p distinct values per dimension, the maximum number of cells is the Cartesian product of all dimensions:

$$p \times p \times \cdots \times p = p^n$$

Question 4 (4%)

Let data cube C consist of 3 dimensions, product, customer and time, and one measure, sale. Assume that there are no concept hierarchies. What is the correct query to find the total sales for product $p7$ by customer $c7$ for all time?

- a. $C(*, *, *)$
- b. $C(p7, *, *)$
- c. $C(p7, c7, *)$
- d. $C(*, c7, *)$

Solution

Answer: c. $C(p7, c7, *)$

We want to find sales for a specific product ($p7$), a specific customer ($c7$), but aggregated over all time. The “*” represents aggregation over that dimension. So we fix product= $p7$, customer= $c7$, and aggregate over time with “*”.

Question 5 (4%)

For the data cube C of question 5, assume the following query: “find the total sales for all products and customers per day”. Which of the following cuboids, if materialized, would you use to answer this query the fastest?

- a. (product, customer)
- b. (time)
- c. (product, customer, time)
- d. 0

Solution

Answer: b. (time)

The query asks for total sales per day (time), aggregating over all products and customers. The cuboid (time) already has pre-computed aggregations for all products and customers for each time value, making it the fastest to answer this query.

Question 6 (4%)

Relative to the MapReduce computation model, which of the following statements is correct?

- a. A MapReduce job splits the input data into independent chunks which are processed by the map tasks in a completely parallel manner
- b. The MapReduce framework operates on $\langle \text{key}, \text{value} \rangle$ pairs
- c. Applications typically implement the Mapper and Reducer functions to provide the map and reduce methods
- d. All of the above

Solution

Answer: d. All of the above

All three statements are correct:

- MapReduce splits input into chunks processed in parallel by map tasks
- MapReduce operates on key-value pairs
- Applications implement Mapper and Reducer functions

Question 7 (4%)

What is the process that schedules MapReduce jobs?

- a. Namenode daemon
- b. Jobtracker
- c. Tasktracker
- d. Datanode daemon

Solution

Answer: b. Jobtracker

The JobTracker is the service that manages MapReduce jobs. It schedules jobs, monitors TaskTrackers, and re-executes failed tasks. (Note: In YARN-based Hadoop 2.x, this role is handled by the ResourceManager and ApplicationMaster.)

Question 8 (4%)

What is the correct sequence of data flow in MapReduce (1=Mapper, 2=Combiner, 3=Reducer, 4=Partitioner)?

- a. 1, 2, 3, 4
- b. 1, 2, 4, 3
- c. 1, 3, 2, 4
- d. 1, 3, 2, 4

Solution

Answer: b. 1, 2, 4, 3

The correct data flow in MapReduce is:

1. **Mapper (1)**: Processes input and emits key-value pairs
2. **Combiner (2)**: Optional local aggregation on mapper output (mini-reducer)
3. **Partitioner (4)**: Determines which reducer receives which keys
4. **Reducer (3)**: Final aggregation of values for each key

Question 9 (4%)

Using the min-max normalization method to $[0, 1]$ to normalize the following group of data: 200, 300, 400, 600, 1000, what are their new values?

- a. 0, 0.2, 0.3, 0.5, 1
- b. 0, 0.2, 0.25, 0.4, 1
- c. 0, 0.125, 0.25, 0.5, 1
- d. 0.2, 0.3, 0.4, 0.6, 1

Solution

Answer: c. 0, 0.125, 0.25, 0.5, 1

Min-max normalization formula: $v' = \frac{v - \min}{\max - \min}$

With $\min = 200$ and $\max = 1000$:

$$\begin{aligned} 200 &\rightarrow \frac{200 - 200}{1000 - 200} = \frac{0}{800} = 0 \\ 300 &\rightarrow \frac{300 - 200}{800} = \frac{100}{800} = 0.125 \\ 400 &\rightarrow \frac{400 - 200}{800} = \frac{200}{800} = 0.25 \\ 600 &\rightarrow \frac{600 - 200}{800} = \frac{400}{800} = 0.5 \\ 1000 &\rightarrow \frac{1000 - 200}{800} = \frac{800}{800} = 1 \end{aligned}$$

Question 10 (4%)

Using the normalization by decimal scaling method to normalize the following group of data: -997, 3, 83, what are their new values?

- a. -1, 0, 0.1
- b. -9.97, 0.03, 0.83
- c. -0.997, 0.003, 0.083
- d. -99.7, 0.3, 8.3

Solution

Answer: c. -0.997, 0.003, 0.083

Decimal scaling normalizes by dividing by a power of 10 such that the maximum absolute value becomes less than 1.

The maximum absolute value is $|-997| = 997$. We need 10^j where $997/10^j < 1$, so $j = 3$ (divide by 1000).

$$-997 \rightarrow -997/1000 = -0.997$$

$$3 \rightarrow 3/1000 = 0.003$$

$$83 \rightarrow 83/1000 = 0.083$$

Question 11 (2%)

What is a relation in RDBMS?

- a. Table
- b. Key
- c. Data Types
- d. Row

Solution

Answer: a. Table

In relational database theory, a relation is represented as a table. Each table (relation) consists of rows (tuples) and columns (attributes).

Question 12 (2%)

Which of the following can replace the below query?

```
SELECT name, course_id
FROM instructor, teaches
WHERE instructor_ID = teaches_ID;
```

- a. SELECT name, course_id FROM instructor NATURAL JOIN teaches;
- b. SELECT name, course_id FROM teaches, instructor WHERE instructor_id=course_id;
- c. SELECT name, course_id FROM instructor;
- d. SELECT course_id FROM instructor JOIN teaches;

Solution

Answer: a. SELECT name, course_id FROM instructor NATURAL JOIN teaches;

A NATURAL JOIN automatically joins tables on columns with the same name. If instructor_ID and teaches_ID refer to the same attribute (both named the same), NATURAL JOIN would produce the same result as the explicit WHERE condition.

Question 13 (2%)

What does the following query do?

```
UPDATE student
SET marks = marks*1.20;
```

- a. Increases marks by 120%
- b. Decreases marks by 20%
- c. Increase marks by 20%
- d. None of the above

Solution

Answer: c. Increase marks by 20%

Multiplying by 1.20 means: $\text{new_marks} = \text{marks} \times 1.20 = \text{marks} + 0.20 \times \text{marks}$
This increases the marks by 20% (not 120%).

Question 14 (2%)

The Select command is a part of what type of statement?

- a. DML
- b. DDL
- c. View
- d. None of the above

Solution

Answer: a. DML

SELECT is a Data Manipulation Language (DML) statement. DML includes SELECT, INSERT, UPDATE, DELETE. DDL (Data Definition Language) includes CREATE, ALTER, DROP.

Question 15 (2%)

The primary key must be?

- a. Unique
- b. Not Null
- c. Both A and B
- d. None of the above

Solution

Answer: c. Both A and B

A primary key must satisfy two constraints:

- **Unique:** No two rows can have the same primary key value
- **Not Null:** The primary key cannot contain NULL values

Question 16 (2%)

How can you change “Thomas” into “Michel” in the “LastName” column in the Users table?

- a. UPDATE User SET LastName = 'Thomas' INTO LastName = 'Michel'
- b. MODIFY Users SET LastName = 'Michel' WHERE LastName = 'Thomas'
- c. MODIFY Users SET LastName = 'Thomas' INTO LastName = 'Michel'
- d. UPDATE Users SET LastName = 'Michel' WHERE LastName = 'Thomas'

Solution

Answer: d. UPDATE Users SET LastName = 'Michel' WHERE LastName = 'Thomas'

The correct SQL UPDATE syntax is:

```
UPDATE table_name SET column = new_value WHERE condition;
```

Question 17 (2%)

Which type of JOIN is used to returns rows that do not have matching values?

- a. Natural JOIN
- b. Outer JOIN
- c. EQUI JOIN
- d. All of the above

Solution

Answer: b. Outer JOIN

An OUTER JOIN returns all rows from one or both tables, including rows that don't have matching values in the other table (with NULLs for missing values). NATURAL JOIN and EQUI JOIN only return matching rows.

Question 18 (2%)

A CASE SQL statement is _____?

- a. A way to establish a loop in SQL
- b. A way to establish an IF-THEN-ELSE in SQL
- c. A way to establish a data definition in SQL
- d. All of the above

Solution

Answer: b. A way to establish an IF-THEN-ELSE in SQL

The CASE statement in SQL provides conditional logic similar to IF-THEN-ELSE:

```
CASE WHEN condition1 THEN result1
      WHEN condition2 THEN result2
      ELSE default_result
END
```

Question 19 (2%)

What is the difference between a PRIMARY KEY and a UNIQUE KEY?

- a. Primary key can store null value, whereas a unique key cannot store null value
- b. We can have only one primary key in a table while we can have multiple unique keys
- c. Primary key cannot be a date variable whereas unique key can be
- d. None of these

Solution

Answer: b. We can have only one primary key in a table while we can have multiple unique keys

Key differences:

- A table can have only ONE primary key but MULTIPLE unique keys
- Primary key cannot contain NULL; unique key can contain NULL (usually one)
- Primary key creates a clustered index by default

Question 20 (2%)

Find the cities name with the condition and temperature from table 'weather' where condition = 'sunny' or 'cloudy' but temperature $\geq$ 60.

- SELECT city, temperature, condition FROM weather WHERE condition = 'cloudy' AND condition = 'sunny' OR temperature $\geq$ 60
- SELECT city, temperature, condition FROM weather WHERE condition = 'cloudy' OR condition = 'sunny' OR temperature $\geq$ 60
- SELECT city, temperature, condition FROM weather WHERE condition = 'cloudy' OR condition = 'sunny' AND temperature $\geq$ 60
- SELECT city, temperature, condition FROM weather WHERE condition = 'cloudy' AND condition = 'sunny' AND temperature $\geq$ 60

Solution

Answer: c. SELECT city, temperature, condition FROM weather WHERE condition = 'cloudy' OR condition = 'sunny' AND temperature $\geq$ 60

We need: (condition is sunny OR cloudy) AND (temperature $\geq$ 60).

Due to operator precedence (AND before OR), option c with proper parentheses interpretation gives: condition = 'cloudy' OR (condition = 'sunny' AND temperature $\geq$ 60).

Note: The question is somewhat ambiguous. Ideally it should be:

```
WHERE (condition = 'sunny' OR condition = 'cloudy')
AND temperature >= 60
```

Question 21 (2%)

Which of the following statement is correct to display all the cities with the condition, temperature, and humidity whose humidity is in the range of 60 to 75 from the 'weather' table?

- SELECT * FROM weather WHERE humidity IN (60 to 75)
- SELECT * FROM weather WHERE humidity BETWEEN 60 AND 75
- SELECT * FROM weather WHERE humidity NOT IN (60 AND 75)
- SELECT * FROM weather WHERE humidity NOT BETWEEN 60 AND 75

Solution

Answer: b. SELECT * FROM weather WHERE humidity BETWEEN 60 AND 75

The BETWEEN operator selects values within a given range (inclusive):

`column BETWEEN value1 AND value2`

This is equivalent to: `column >= 60 AND column <= 75`

Question 22 (2%)

Which statement is used to get all data from the student table whose name starts with p?

- a. `SELECT * FROM student WHERE name LIKE '%p%';`
- b. `SELECT * FROM student WHERE name LIKE 'p%';`
- c. `SELECT * FROM student WHERE name LIKE '_p%';`
- d. `SELECT * FROM student WHERE name LIKE '%p';`

Solution

Answer: b. SELECT * FROM student WHERE name LIKE 'p%';

In SQL LIKE patterns:

- `%` matches any sequence of characters
- `_` matches exactly one character

`'p%'` matches strings that START with 'p' followed by anything.

Question 23 (2%)

Which of the SQL statements is correct?

- a. `SELECT Username AND Password FROM Users`
- b. `SELECT Username, Password FROM Users`
- c. `SELECT Username, Password WHERE Username = 'user1'`
- d. None of these

Solution

Answer: b. SELECT Username, Password FROM Users

The correct SELECT syntax requires columns separated by commas (not AND) and a FROM clause specifying the table.

Question 24 (2%)

Which SQL keyword is used to retrieve only unique values?

- A. DISTINCTIVE

- B. UNIQUE
- C. DISTINCT
- D. DIFFERENT

Solution

Answer: C. DISTINCT

The DISTINCT keyword eliminates duplicate rows from query results:

```
SELECT DISTINCT column_name FROM table_name;
```

Question 25 (2%)

Which of the following are valid logical operators in SQL?

- A. SOME
- B. ALL
- C. AND
- D. All of the above

Solution

Answer: D. All of the above

All are valid SQL logical operators:

- **SOME/ANY:** Compares a value to each value in a list or subquery
- **ALL:** Compares a value to all values in a list or subquery
- **AND:** Combines two boolean expressions

Question Set 2

Answer to the following questions. Every question is worth 10% of the total grade.

Question 1 (10%)

Consider the following global schema and local schema:

Global schema

Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type)

Local schema

Loan-Accounts (loan-account-number, holder-name, amount)

Loan-Payments (loan-account-number, payment-number, payment-amount, date)

It is requested to create the Global-As-View (GAV) and Local-As-View (LAV) mappings for the above schemas under the ‘open-world’ assumption. Attributes should be matched only if they have the exact same name.

Which of the following are true (note: more than one or none of the statements may be true):

- a. The following GAV mapping holds:
Bank-Recent-Payments (NULL, holder-name, payment-amount, NULL) $\supseteq$ Loan-Accounts (loan-account-number, holder-name, amount), Loan-Payments (loan-account-number, payment-number, payment-amount, date)
- b. The following GAV mapping holds:
Bank-Recent-Payments (NULL, holder-name, payment-amount, NULL) $\subseteq$ Loan-Accounts (loan-account-number, holder-name, amount), Loan-Payments (loan-account-number, payment-number, payment-amount, date)
- c. The following GAV mapping holds:
Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type) $\subseteq$ Loan-Accounts (loan-account-number, holder-name, amount), Loan-Payments (loan-account-number, payment-number, payment-amount, date)
- d. The following GAV mapping holds:
Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type) $\supseteq$ Loan-Accounts (loan-account-number, holder-name, amount), Loan-Payments (loan-account-number, payment-number, payment-amount, date)
- e. The following LAV mapping holds:
Loan-Accounts (NULL, holder-name, NULL) $\supseteq$ Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type)
- f. The following LAV mapping holds:
Loan-Accounts (loan-account-number, holder-name, amount) $\supseteq$ Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type)
- g. The following LAV mapping holds:
Loan-Accounts (NULL, holder-name, NULL) $\subseteq$ Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type)
- h. The following LAV mapping holds:
Loan-Payments (NULL, NULL, payment-amount, NULL) $\subseteq$ Bank-Recent-Payments (account-number, holder-name, payment-amount, account-type)

Solution

Answer: b, g, h

In data integration:

- **GAV (Global-As-View):** The global schema is defined as a view over local schemas
- **LAV (Local-As-View):** Local schemas are defined as views over the global schema
- Under **open-world assumption**: data may exist that is not in sources

Matching attributes (same name): **holder-name, payment-amount**

Analysis:

GAV mappings: The global relation should be expressed using local relations.

- The join of Loan-Accounts and Loan-Payments on loan-account-number provides: holder-name (from Loan-Accounts) and payment-amount (from Loan-Payments)
- Under open-world assumption, the local sources may have incomplete data
- **Option b is correct:** GAV with $\subseteq$ means global is contained in what we can derive from local sources (with NULLs for unmatched attributes)

LAV mappings: Each local relation is expressed as a view over the global schema.

- **Option g:** Loan-Accounts projected to (NULL, holder-name, NULL) $\subseteq$ Bank-Recent-Payments projected to holder-name. This is valid as holder-name from Loan-Accounts can be found in Bank-Recent-Payments.
- **Option h:** Loan-Payments projected to payment-amount $\subseteq$ Bank-Recent-Payments projected to payment-amount. Valid mapping.

Question 2 (10%)

Assume the following conjunctive queries:

Q1: $h(X, Y) :- r(X, W), r(Z, Y), g(W, Z), g(Z, W)$

Q2: $h(U, V) :- r(U, K), g(K, K), r(K, V)$

Which of the following are true (note: more than one or none of the statements may be true):

- Q2 is not contained in Q1 because there is no containment mapping from Q1 to Q2
- Q1 is not contained in Q2 because there is no containment mapping from Q2 to Q1
- Q2 is contained in Q1 because there is containment mapping from Q1 to Q2
- Q1 and Q2 are equivalent
- Q2 is contained in Q1 although there is no containment mapping from Q1 to Q2 but there is containment of Q1 in Q1 and this can be proved with canonical databases
- There is no containment mapping from Q1 to Q2, and canonical databases can prove that there is no containment of Q1 in Q2
- Both the techniques of containment mapping and canonical databases can prove that there is no containment of Q1 in Q2

Solution

Answer: c, f (or a, b depending on interpretation)

Query Containment Analysis:

Q1: $h(X, Y) :- r(X, W), r(Z, Y), g(W, Z), g(Z, W)$

Q2: $h(U, V) :- r(U, K), g(K, K), r(K, V)$

Checking $Q2 \subseteq Q1$ (containment mapping from Q1 to Q2):

We need a mapping θ from variables of Q1 to variables of Q2 such that:

- $\theta(\text{head of Q1}) = \text{head of Q2}$: $\theta(X) = U, \theta(Y) = V$
- $\theta(\text{body of Q1}) \subseteq \text{body of Q2}$

Try: $X \rightarrow U, Y \rightarrow V, W \rightarrow K, Z \rightarrow K$

Check body:

- $r(X, W) \rightarrow r(U, K) \checkmark$
- $r(Z, Y) \rightarrow r(K, V) \checkmark$
- $g(W, Z) \rightarrow g(K, K) \checkmark$
- $g(Z, W) \rightarrow g(K, K) \checkmark$

A valid containment mapping exists! So $Q2 \subseteq Q1$, meaning **c is correct**.

Checking $Q1 \subseteq Q2$:

Q2's body requires $g(K, K)$ where both arguments are the same. Q1 has $g(W, Z)$ and $g(Z, W)$ but doesn't force $W = Z$. The canonical database of Q1 would have distinct values for W and Z, but Q2 requires the same value in $g(K, K)$. No containment mapping exists from Q2 to Q1, and canonical databases confirm $Q1 \not\subseteq Q2$.

So **f is correct**: canonical databases can prove there is no containment of Q1 in Q2.

Question 3 (10%)

The $\text{DISTINCT}(X)$ operator is used to return only distinct (unique) values for datatype (or column) X in the entire dataset. As an example, for the dataset with ZIPCODEs (12345, 12345, 78910, 78910, 78910), $\text{DISTINCT}(\text{ZIPCODE}) = (12345, 78910)$. Provide algorithm pseudocode to implement the $\text{DISTINCT}(X)$ operator using Map-Reduce.

Hint: assume the input comprises of (id, X) records

Solution

MapReduce Algorithm for DISTINCT(X):

The key insight is that MapReduce's shuffle phase naturally groups all identical keys together, and we only need to emit each unique value once.

Algorithm 1 DISTINCT(X) using MapReduce

```
1: Map Function:
2: Input: (id, X) – a record with identifier and value X
3: Output: (X, null) – emit X as key, no value needed
4:
5: function map(id, X):
6:     emit(X, null)
7:
8: Reduce Function:
9: Input: (X, [null, null, ...]) – X with list of null values
10: Output: (X, null) – emit each unique X exactly once
11:
12: function reduce(X, values[]):
13:     emit(X, null) {Output X only once, ignore duplicate values}
```

Explanation:

1. **Map Phase:** For each input record (id, X), emit the value X as the key. The actual value part can be null/empty since we only care about the keys.
2. **Shuffle Phase:** MapReduce automatically groups all records with the same key together. All duplicate X values will be sent to the same reducer.
3. **Reduce Phase:** For each unique key X, the reducer is called once. We simply emit X once, effectively removing all duplicates.

Example with ZIPCODE data:

Input: (1, 12345), (2, 12345), (3, 78910), (4, 78910), (5, 78910)

Map output:

(12345, null), (12345, null), (78910, null), (78910, null), (78910, null)

After Shuffle (grouped by key):

12345 -> [null, null]

78910 -> [null, null, null]

Reduce output:

(12345, null)

(78910, null)

Final result: DISTINCT(ZIPCODE) = {12345, 78910}

Optional Optimization with Combiner:

A combiner can be used to reduce network traffic by performing local deduplication:

- ```
1: function combine(X, values[]):
2: emit(X, null) {Local deduplication before shuffle}
```

This reduces the amount of data transferred during the shuffle phase from  $O(n)$  to  $O(\text{unique values per mapper})$ .

## Answer Key - Question Set 1

| Q# | Answer | Q# | Answer |
|----|--------|----|--------|
| 1  | b      | 14 | a      |
| 2  | b      | 15 | c      |
| 3  | a      | 16 | d      |
| 4  | c      | 17 | b      |
| 5  | b      | 18 | b      |
| 6  | d      | 19 | b      |
| 7  | b      | 20 | c      |
| 8  | b      | 21 | b      |
| 9  | c      | 22 | b      |
| 10 | c      | 23 | b      |
| 11 | a      | 24 | C      |
| 12 | a      | 25 | D      |
| 13 | c      |    |        |